CHƯƠNG 1

KIẾN TRÚC MÁY TÍNH VÀ NGUYÊN LÝ VON NEUMANN

Nền tảng phần cứng và phần mềm • Chương 1

Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology



Từ chương trình lưu trữ đến chu kỳ lệnh và các lớp trừu tượng lập trình.

01

HỆ THỐNG MÁY TÍNH

Máy tính là thiết bị có thể lập trình, hình thành từ sự phối hợp giữa phần cứng và phần mềm.

Chương 1 • Kiến trúc máy tính • Nguyên lý Von Neumann

Mục tiêu học tập

Sau chương này, sinh viên có thể...

1. **Giải thích mô hình Von Neumann**
   Trình bày kiến trúc và lý do mô hình vẫn là nền tảng của máy tính hiện đại.

2. **Mô tả phần cứng cơ bản**
   Nêu chức năng CPU, bộ nhớ chính, thiết bị vào/ra và bus kết nối.

3. **Phân tích chu kỳ lệnh**
   Theo dõi Fetch–Decode–Execute và đọc chương trình hợp ngữ đơn giản.

4. **Phân biệt RISC và CISC**
   Nhận biết hai hướng thiết kế kiến trúc tập lệnh ISA chính.

5. **Nhận diện các lớp trừu tượng**
   Kết nối phần cứng, ISA, hệ điều hành, hợp ngữ và ngôn ngữ bậc cao.

Lộ trình của chương

Bốn phần nội dung theo đúng trình tự của tài liệu nguồn.

1

Tổng quan

Hệ thống máy tính + lịch sử

2

Von Neumann

Mô hình + phần cứng

3

Thực thi

Stored-program + chu kỳ lệnh

4

ISA và lập trình

RISC/CISC + các mức trừu tượng

Máy tính hiện đại là thiết bị có thể lập trình

Chương trình phần mềm quyết định hành vi của cùng một phần cứng.

PROGRAMMABLE

Máy tính không bị giới hạn ở một tập thao tác cố định; chương trình được nạp vào xác định nhiệm vụ mà hệ thống thực hiện.

Đầu vào

Dữ liệu và lệnh được đưa vào hệ thống.

Xử lý

Phần cứng thực thi chuỗi lệnh của chương trình.

Kết quả

Thông tin được lưu trữ, hiển thị hoặc truyền đi.

Phần cứng và phần mềm không thể tách rời

Khả năng xử lý hình thành từ sự phối hợp của hai thành phần.

PHẦN CỨNG — HARDWARE

Các thành phần vật lý: CPU, mô-đun bộ nhớ, thiết bị lưu trữ, thiết bị vào/ra và bus kết nối.

Vai trò: cung cấp khả năng thực thi và xử lý dữ liệu.

PHẦN MỀM — SOFTWARE

Tập hợp chương trình và tập lệnh điều khiển phần cứng thực hiện nhiệm vụ cụ thể.

Vai trò: xác định chuỗi thao tác và hành vi của hệ thống.

Phần cứng quyết định máy có thể làm gì; phần mềm quyết định máy sẽ làm gì.

Sáu lớp trừu tượng che giấu dần độ phức tạp

Mỗi lớp cung cấp một giao diện rõ ràng cho lớp bên trên.

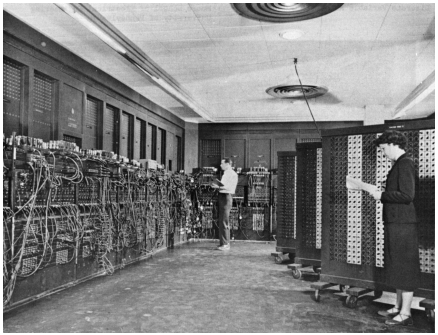
Ngôn ngữ bậc cao	Hàm, đối tượng, lớp
Hợp ngữ	Mnemonic gần với ISA
Hệ điều hành	Tiến trình, bộ nhớ ảo, tệp
ISA	Lệnh, thanh ghi, kiểu dữ liệu
Vi kiến trúc	ALU, datapath, pipeline
Mạch số	Transistor và cổng logic

Cùng một

ISA có thể được hiện thực bởi nhiều vi kiến trúc khác nhau.

ENIAC mở đầu kỷ nguyên máy tính điện tử số

Hiệu năng đột phá, nhưng việc đổi chương trình vẫn cần tái cấu hình vật lý.



US Army / Wikimedia Commons, Public Domain.

Tháng 5/1945

ENIAC được hoàn thành tại Đại học Pennsylvania.

10/12/1945

Máy chạy chương trình đầu tiên.

Khoảng 5.000 phép cộng/giây

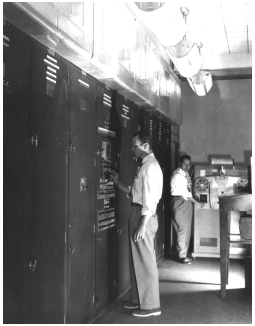
Tốc độ chưa từng có vào thời điểm đó.

Hạn chế

Đi dây và đặt công tắc có thể mất nhiều ngày hoặc nhiều tuần.

Chương trình lưu trữ thay đổi cách máy tính được lập trình

EDVAC, Manchester Baby và EDSAC đánh dấu bước chuyển lịch sử.



EDVAC — US Army / Wikimedia Commons, Public Domain.

1945

Báo cáo kỹ thuật “First Draft of a Report on the EDVAC”.

1948

Manchester Baby chạy chương trình lưu trữ đầu tiên.

1949

EDVAC được bàn giao; EDSAC trở thành hệ thống thực dụng sớm.

Sau đó

IBM 701, System/360, x86, ARM và RISC-V tiếp tục phát triển.

Bốn thế hệ máy tính phản ánh công nghệ chế tạo

Mỗi bước tiến linh kiện làm tăng mật độ, độ tin cậy và khả năng xử lý.

Thế hệ	Năm	Công nghệ	Ví dụ
Thế hệ 1	1940–1955	Bóng đèn điện tử	ENIAC, EDVAC
Thế hệ 2	1955–1965	Transistor	IBM 1401, UNIVAC
Thế hệ 3	1965–1980	Mạch tích hợp (IC)	IBM System/360
Thế hệ 4	1980–nay	Vi mạch VLSI	Intel x86, ARM, RISC-V

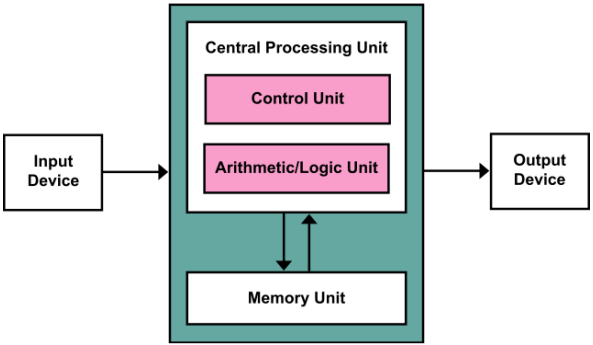
02

KIẾN TRÚC VON NEUMANN

Lệnh và dữ liệu cùng nằm trong bộ nhớ; CPU thực thi chương trình theo chu kỳ điều khiển.

Mô hình Von Neumann tổ chức hệ thống quanh CPU và bộ nhớ

Một bus chung kết nối xử lý, lưu trữ và vào/ra.



Wikimedia Commons, CC BY-SA 3.0.

CPU

Control Unit điều phối; ALU thực hiện phép toán.

Memory

Lưu cả lệnh và dữ liệu dưới dạng nhị phân.

Input / Output

Giao tiếp với bàn phím, màn hình, ổ đĩa và mạng.

System bus

Kết nối CPU, memory và hệ thống I/O trong mô hình tổng quát.

Bốn thành phần phối hợp để thực thi chương trình

Mỗi khối đảm nhiệm một vai trò riêng trong đường đi của lệnh và dữ liệu.

01 MEMORY

Lưu trữ lệnh và dữ liệu trong các ô nhớ có địa chỉ.

02 ALU

Thực hiện phép toán số học, logic và so sánh.

03 CONTROL UNIT

Nạp, giải mã và phát tín hiệu điều khiển các thành phần.

04 I/O

Trao đổi dữ liệu với thiết bị và môi trường bên ngoài.

Ba nguyên lý cốt lõi tạo nên mô hình Von Neumann

Các nguyên lý bổ trợ lẫn nhau để phần mềm điều khiển phần cứng.

1 Phân rã hệ thống

Máy tính gồm bộ nhớ, ALU, đơn vị điều khiển và I/O phối hợp qua các đường kết nối.

2 Chương trình lưu trữ

Lệnh và dữ liệu cùng được biểu diễn nhị phân và lưu trong bộ nhớ có thể định địa chỉ.

3 Thực thi tuần tự

Lệnh được nạp và thực thi theo thứ tự, trừ khi branch hoặc jump thay đổi PC.

Có thể đổi hành vi của máy bằng cách nạp chương trình mới, không cần đi dây lại phần cứng.

Khái niệm chương trình lưu trữ tạo nền móng cho phần mềm

Chương trình có thể được nạp, lưu, sao chép và xử lý như dữ liệu.

Hệ điều hành

Một chương trình đặc biệt quản lý các chương trình khác.

Trình biên dịch

Đọc mã nguồn như dữ liệu và tạo ra mã máy để thực thi.

Self-modifying code

Chương trình có thể thay đổi chính lệnh của nó tại runtime.

Công nghiệp phần mềm

Ngôn ngữ, thư viện và ứng dụng dựa trên khả năng thao tác chương trình.

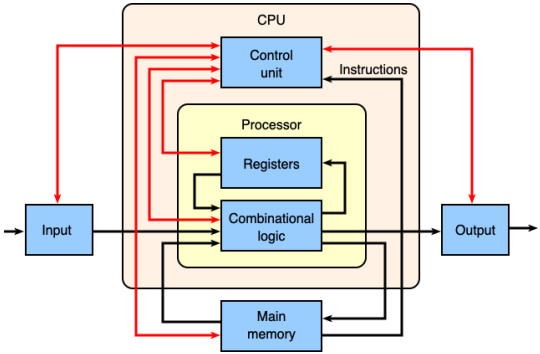
03

PHẦN CỨNG CƠ BẢN

CPU thực thi lệnh, bộ nhớ giữ trạng thái và I/O kết nối hệ thống với thế giới bên ngoài.

CPU là trung tâm thực thi lệnh của hệ thống

Control Unit, ALU và thanh ghi được tích hợp trên bộ xử lý.



Wikimedia Commons, CC BY-SA 4.0.

Control Unit

Nạp và giải mã lệnh; tạo tín hiệu điều khiển.

ALU

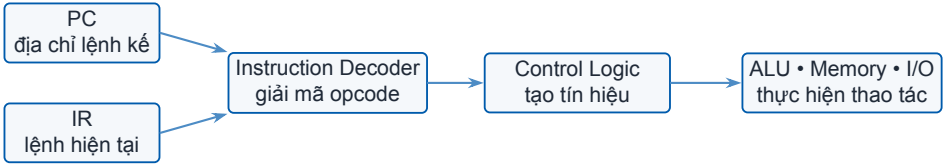
Thực hiện tính toán số học, logic và so sánh.

Registers

Giữ toán hạng, địa chỉ và kết quả trung gian với tốc độ rất cao.

Đơn vị điều khiển điều phối toàn bộ chu kỳ lệnh

CU không trực tiếp tính toán; nó ra lệnh cho datapath, memory và I/O.



Program Counter — PC

Giữ địa chỉ lệnh tiếp theo; branch và jump thay đổi giá trị PC.

Instruction Register — IR

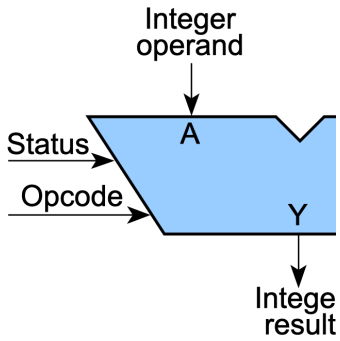
Giữ lệnh vừa nạp trong suốt quá trình giải mã và thực thi.

Instruction Decoder

Diễn giải opcode và tạo chuỗi tín hiệu điều khiển tương ứng.

ALU là lõi tính toán của CPU

ALU nhận toán hạng và tín hiệu điều khiển, sau đó tạo kết quả cùng các cờ trạng thái.



Wikimedia Commons, CC0 — Public Domain.

Phép toán số học

Cộng, trừ, nhân và chia số nguyên; các phép phức tạp có thể do FPU hoặc vector unit đảm nhận.

Phép toán logic

AND, OR, NOT, XOR theo bit và các phép so sánh.

Đầu ra trạng thái

Zero, carry, negative và overflow hỗ trợ các lệnh rẽ nhánh có điều kiện.

Thanh ghi cung cấp vùng lưu trữ nhanh nhất

Toán hạng và kết quả trung gian được giữ ngay trên chip xử lý.

General-purpose registers

R0, R1, R2, ... giữ toán hạng và kết quả tạm thời. Số lượng phụ thuộc ISA: MIPS có 32 GPR, x86-64 có 16 thanh ghi kiến trúc.

Status flags

Các cờ zero, carry, negative và overflow ghi nhận trạng thái của phép toán gần nhất.

Register renaming

CPU hiện đại thường có nhiều thanh ghi vật lý hơn thanh ghi kiến trúc để hỗ trợ thực thi hiệu quả.

Pipeline chồng lấp nhiều lệnh trên năm giai đoạn

Pipeline tăng throughput, không làm một lệnh riêng lẻ hoàn thành nhanh hơn.



Khi pipeline đã đầy, trường hợp lý tưởng hoàn tất một lệnh trong mỗi chu kỳ.

Ba loại hazard làm gián đoạn pipeline

Mỗi xung đột cần một cơ chế xử lý khác nhau.

Structural hazard

Phần cứng không hỗ trợ đồng thời mọi yêu cầu.

Ví dụ: một cổng bộ nhớ phải phục vụ cả thao tác nạp lệnh và truy cập dữ liệu.

Data hazard

Một lệnh cần kết quả mà lệnh trước chưa ghi xong.

Ứng phó: stalling hoặc operand forwarding.

Control hazard

Branch làm gián đoạn luồng nạp tuần tự.

Ứng phó: branch prediction, ví dụ bộ đếm bão hòa hai bit.

CPU hiện đại mở rộng vượt ra ngoài pipeline cơ bản

Hiệu năng tăng nhờ khai thác song song ở nhiều cấp độ.

Superscalar

Phát nhiều lệnh mỗi chu kỳ tới các đơn vị thực thi song song.

Out-of-order

Thực thi lệnh khi toán hạng sẵn sàng thay vì chờ đúng thứ tự chương trình.

SIMD

Một lệnh tác động lên nhiều phần tử: SSE, AVX, AVX-512 hoặc NEON.

Multi-core

Tích hợp nhiều lõi để vượt giới hạn memory wall, ILP wall và power wall.

Định luật Amdahl: phần tuần tự giới hạn tốc độ tăng tốc tối đa của chương trình.

Bộ nhớ là một mảng tuyến tính các ô có địa chỉ

Trong hệ byte-addressable, mỗi địa chỉ xác định một byte 8 bit.



N bit địa chỉ

Có thể định địa chỉ tối đa `2^N` ô nhớ.

Hệ thống 32-bit

Không gian địa chỉ gồm `2^32` byte, xấp xỉ 4 GB.

Hệ thống 64-bit

Giới hạn lý thuyết đạt tới 16 exabyte.

Fetch và Store là hai thao tác bộ nhớ nguyên thủy

MAR mang địa chỉ; MDR mang dữ liệu giữa CPU và memory.

FETCH(address)

Đọc nội dung tại địa chỉ được chỉ định mà không phá hủy giá trị cũ.

Đường đi: address → MAR → Memory → MDR → CPU.

STORE(address, value)

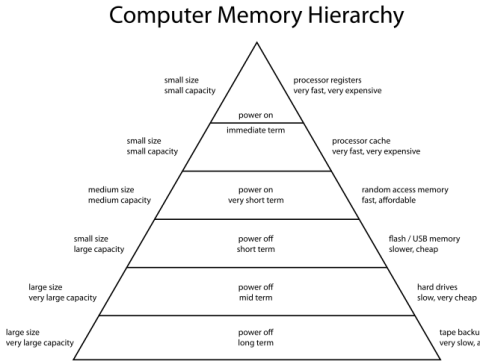
Ghi một giá trị mới vào ô nhớ và thay thế nội dung trước đó.

Đường đi: address → MAR; value → MDR → Memory.



Phân cấp bộ nhớ cân bằng tốc độ, dung lượng và chi phí

Mức càng gần CPU càng nhanh, nhỏ và đắt trên mỗi bit.



Wikimedia Commons, Public Domain.

Registers

Nhanh nhất; dung lượng nhỏ nhất; nằm trong CPU.

Cache L1 / L2 / L3

SRAM trên chip, lưu dữ liệu được dùng gần đây.

Main memory

DRAM dung lượng GB; chậm hơn cache.

SSD / HDD

Dung lượng lớn, không mất dữ liệu khi tắt nguồn, nhưng chậm hơn nhiều.

Các mức bộ nhớ khác nhau nhiều bậc về độ trễ

Số liệu trong chương minh họa đánh đổi giữa tốc độ và dung lượng.

Mức	Loại	Dung lượng	Truy cập	Khả biến
Thanh ghi	Flip-flop	< 1 KB	< 1 ns	Có
Cache L1	SRAM	32–64 KB/lõi	1–4 ns	Có
Cache L2	SRAM	256 KB–1 MB/lõi	3–10 ns	Có
Cache L3	SRAM	8–32 MB	10–20 ns	Có
RAM	DRAM	8–128 GB	50–82 ns	Có
SSD	Flash	256 GB–4 TB	0.1–1 ms	Không
HDD	Đĩa từ	1–20 TB	5–15 ms	Không

Cache khai thác tính cục bộ của chương trình

Dữ liệu có khả năng được dùng lại hoặc được truy cập theo vùng lân cận.

TEMPORAL LOCALITY

Nếu một địa chỉ vừa được truy cập, địa chỉ đó có khả năng được truy cập lại trong tương lai gần.

Ví dụ: cùng một biến được đọc và ghi nhiều lần trong vòng lặp.

SPATIAL LOCALITY

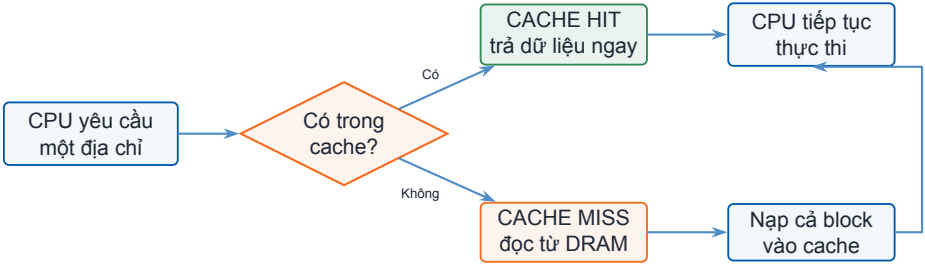
Nếu một địa chỉ vừa được truy cập, các địa chỉ lân cận có khả năng được truy cập sớm.

Ví dụ: duyệt tuần tự các phần tử liên tiếp của một mảng.

Cache truyền dữ liệu theo block/cache line, thường chứa nhiều byte liên tiếp.

Cache hit trả dữ liệu nhanh; cache miss phải chờ DRAM

Hit rate là một thông số quyết định hiệu năng của hệ thống.



Một cache miss có thể làm CPU chờ hàng trăm chu kỳ xung nhịp.

SRAM và DRAM phục vụ hai vị trí khác nhau

Tốc độ và mật độ lưu trữ quyết định nơi mỗi công nghệ được sử dụng.

SRAM — CACHE

Dùng 4–6 transistor để tạo flip-flop ổn định.

Không cần làm mới; nhanh nhưng mật độ thấp và chi phí cao.

Phù hợp cho cache L1, L2 và L3.

DRAM — MAIN MEMORY

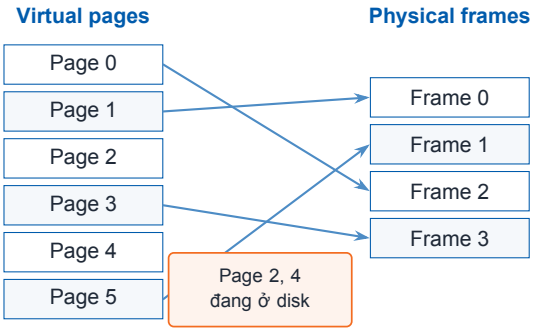
Mỗi bit dùng một transistor và một tụ điện.

Cần làm mới định kỳ khoảng 64 ms; mật độ cao và chi phí thấp.

Phù hợp cho bộ nhớ chính dung lượng lớn.

Bộ nhớ ảo tách địa chỉ chương trình khỏi RAM vật lý

Page table ánh xạ virtual page sang physical frame.



Dung lượng biểu kiến

Chương trình có thể sử dụng không gian địa chỉ lớn hơn RAM vật lý.

Cô lập

Mỗi tiến trình có không gian địa chỉ riêng, tăng ổn định và bảo mật.

Page fault

Hệ điều hành nạp trang từ disk vào RAM và cập nhật page table.

TLB tăng tốc dịch địa chỉ; thrashing phá hủy hiệu năng

Bộ nhớ ảo hiệu quả khi working set phù hợp với RAM khả dụng.

TLB — Translation Lookaside Buffer

Lưu các ánh xạ địa chỉ ảo–vật lý vừa dùng gần đây.

Đây là bộ nhớ liên kết có tốc độ rất cao; hit rate thường trên 99% nhờ tính cục bộ.

THRASHING

Xảy ra khi tổng working set vượt quá bộ nhớ vật lý.

Hệ thống liên tục đẩy và nạp trang; CPU dành phần lớn thời gian xử lý page fault thay vì tính toán.

Hệ thống I/O kết nối máy tính với thế giới bên ngoài

Bộ điều khiển chuyên dụng giúp CPU không phải chờ mọi thao tác thiết bị.

Thiết bị giao tiếp

Bàn phím, chuột, màn hình và giao diện mạng trao đổi thông tin với người dùng hoặc hệ thống khác.

Thiết bị lưu trữ

HDD, SSD, đĩa quang và băng từ lưu dữ liệu lâu dài.

Khoảng cách tốc độ

RAM: khoảng 50–82 ns.
HDD: khoảng 10 ms.
Ổ đĩa có thể chậm hơn RAM khoảng 200.000 lần.

I/O controller quản lý thiết bị để CPU có thể tiếp tục thực thi công việc khác.

Interrupt và DMA giảm mức can thiệp của CPU

Hai cơ chế phối hợp để truyền dữ liệu hiệu quả.

INTERRUPT

Khi I/O hoàn tất, controller phát tín hiệu ngắt. Tín hiệu này tạm dừng luồng lệnh hiện tại và kích hoạt Interrupt Service Routine (ISR).

DMA — DIRECT MEMORY ACCESS

Dữ liệu được truyền trực tiếp giữa bộ nhớ chính và buffer của I/O controller mà không đi qua CPU, nhờ đó giảm mức can thiệp của CPU.

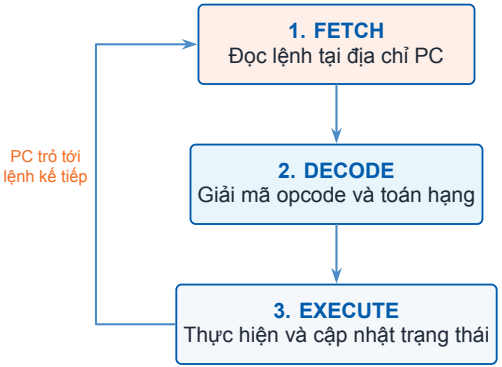
04

CHU KỲ THỰC THI LỆNH

CPU lặp lại Fetch–Decode–Execute cho đến khi gặp lệnh dừng hoặc lỗi nghiêm trọng.

Chu kỳ Fetch–Decode–Execute là nhịp hoạt động của CPU

Mỗi vòng xử lý một lệnh theo mô hình khái niệm.



1 FETCH

Đọc lệnh tiếp theo từ memory dựa trên địa chỉ trong PC.

2 DECODE

Instruction Decoder phân tích opcode và trường toán hạng trong IR.

3 EXECUTE

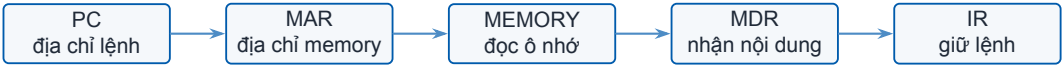
ALU, memory hoặc I/O thực hiện thao tác mà opcode yêu cầu.

REPEAT

PC xác định lệnh kế tiếp; pipeline có thể chồng lấp nhiều chu kỳ.

Pha Fetch chuyển lệnh từ bộ nhớ vào IR

PC, MAR, MDR và IR phối hợp theo một thứ tự xác định.



1 Copy address

Nội dung PC được sao chép vào MAR để chỉ định ô nhớ cần đọc.

2 Read instruction

Memory trả lệnh qua MDR; lệnh được chép vào IR.

3 Advance PC

PC tăng theo độ dài lệnh để trỏ tới lệnh kế tiếp.

Pha Decode xác định thao tác và toán hạng

Instruction Decoder phân tích các trường bit trong IR.

OPCODE

Trường mã lệnh xác định phép toán cần thực hiện: nạp, lưu, cộng, logic, nhảy hoặc gọi hệ thống.

Opcode dài k bit có thể biểu diễn tối đa `2^k` phép toán.

OPERANDS

Các trường còn lại xác định thanh ghi, địa chỉ bộ nhớ, hằng số tức thì hoặc điều kiện.

CPU CISC hiện đại có thể dịch một lệnh thành nhiều micro-op.

Pha Execute phụ thuộc hoàn toàn vào opcode

Thao tác có thể dùng ALU, truy cập memory hoặc thay đổi PC.

LOAD X

Đặt X vào MAR, đọc memory và nạp giá trị vào thanh ghi R.

ADD X

Đọc giá trị tại X, ALU cộng với R và ghi kết quả trở lại R.

JUMP X

Ghi X vào PC; lệnh kế tiếp được nạp từ địa chỉ mới.

JUMPGT X

Chỉ thay đổi PC khi cờ trạng thái cho thấy điều kiện lớn hơn được thỏa mãn.

Các tên trên là lệnh giả định minh họa; mỗi ISA dùng mnemonic và cơ chế cụ thể riêng.

Lệnh máy là một từ nhị phân có cấu trúc

ISA quy định cách phân chia opcode và các trường toán hạng.



Độ dài lệnh

Có thể cố định như LC-3 16-bit hoặc biến đổi như x86 từ 1 đến 15 byte.

Loại toán hạng

Thanh ghi, địa chỉ memory, immediate hoặc điều kiện rẽ nhánh.

Tập lệnh LC-3: số học và truy cập trực tiếp

LC-3 là kiến trúc giả lập 16-bit dùng phổ biến trong giảng dạy.

Opcode	Lệnh	Loại	Mô tả
0000	BRn, BRp, BRz, ...	Rẽ nhánh	Theo cờ N, Z, P
0001	ADD DR, SR1, SR2/imm5	Toán hạng	Cộng register hoặc immediate
0010	LD DR, PCoffset9	Nạp	Memory[PC+offset] → DR
0011	ST SR, PCoffset9	Lưu	SR → Memory[PC+offset]
0100	JSR / JSRR	Nhảy	Gọi subroutine
0101	AND DR, SR1, SR2/imm5	Toán hạng	AND register hoặc immediate
0110	LDR DR, BaseR, offset6	Nạp	Memory[BaseR+offset] → DR
0111	STR SR, BaseR, offset6	Lưu	SR → Memory[BaseR+offset]

Tập lệnh LC-3: gián tiếp, hệ thống và điều khiển

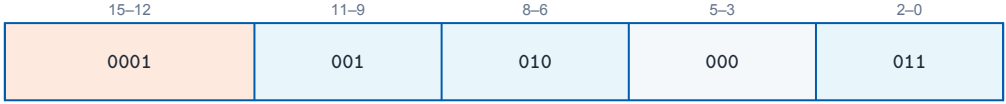
LC-3 định nghĩa 15 lệnh và dành một opcode cho mở rộng.

Opcode	Lệnh	Loại	Mô tả
1000	RTI	Hệ thống	Trở về từ ngắt
1001	NOT DR, SR	Toán hạng	Phủ định bit
1010	LDI DR, PCoffset9	Nạp gián tiếp	Nạp qua địa chỉ trung gian
1011	STI SR, PCoffset9	Lưu gián tiếp	Lưu qua địa chỉ trung gian
1100	JMP / RET	Nhảy	Nhảy qua register
1101	Reserved	–	Dành cho mở rộng tương lai
1110	LEA DR, PCoffset9	Nạp địa chỉ	DR ← PC+offset
1111	TRAP	Hệ thống	IN, OUT, PUTS, HALT

Ví dụ mã hóa một lệnh LC-3 16-bit

Opcode và ba trường register cùng nằm trong một instruction word.

ADD R1, R2, R3



0001 001 010 000 011

RISC và CISC theo đuổi hai triết lý thiết kế ISA

Không thể đánh giá chỉ bằng số chu kỳ của một lệnh riêng lẻ.

RISC — Reduced Instruction Set Computing

Tập lệnh nhỏ và đơn giản; định dạng thường cố định.

Kiến trúc load-store: chỉ load/store truy cập memory; ALU làm việc trên register.

Pipeline thuận lợi. Đại diện: MIPS, SPARC, ARM, RISC-V.

CISC — Complex Instruction Set Computing

Tập lệnh phong phú, nhiều chế độ địa chỉ và độ dài biến đổi.

Mật độ mã cao; một lệnh có thể thực hiện thao tác phức tạp.

x86 hiện đại dịch lệnh thành micro-op trước khi thực thi.

ARM và x86 minh họa khác biệt RISC–CISC

Hiệu năng thực tế còn phụ thuộc compiler, microarchitecture, memory và workload.

Tiêu chí	ARM — RISC	x86 — CISC
Triết lý	Nhỏ gọn, hiệu quả năng lượng	Tương thích ngược từ 1978
Định dạng	Cố định 32-bit; Thumb 16-bit	Biến đổi 1–15 byte
GPR	16	16 trên x86-64
Memory trong ALU	Không — load-store	Có, ví dụ add eax, [ebx]
Mật độ mã	Thấp; Thumb-2 cải thiện	Cao
Lĩnh vực	Di động, nhúng, IoT, máy chủ	PC, máy chủ, trò chơi

05

HỢP NGỮ VÀ LẬP TRÌNH

Các lớp ngôn ngữ cung cấp mức trừu tượng khác nhau trên cùng tập lệnh phần cứng.

Hợp ngữ là dạng ký hiệu gần với lệnh máy

Assembler dịch mnemonic và label thành chuỗi bit mà CPU thực thi.

Mnemonic

Thay opcode nhị phân bằng tên gợi nhớ như LOAD, ADD, STORE hoặc JUMP.

Symbolic labels

Thay địa chỉ số bằng nhãn, giúp mã dễ đọc và giảm lỗi.

ISA-dependent

Hợp ngữ gắn với một kiến trúc cụ thể; mã x86 không chạy trực tiếp trên ARM.

Ứng dụng: device driver, firmware, bootloader, ISR và tối ưu đoạn mã hiệu năng cao.

Ví dụ hợp ngữ: tính A = B + C

Tên lệnh là mã giả minh họa; A, B, C nằm tại địa chỉ 100, 150, 151.

```
; Tinh A = B + C
LOAD   150     ; R <- MEM[150], nap B
ADD    151     ; R <- R + MEM[151]
STORE  100     ; MEM[100] <- R
HALT           ; ket thuc
```

1 LOAD

Đưa B từ memory vào register R.

2 ADD

ALU cộng C với giá trị đang có trong R.

3 STORE

Ghi kết quả R trở lại địa chỉ của A.

Ví dụ hợp ngữ: xuất số lớn hơn

Cấu trúc if–else được phân rã thành compare và các lệnh jump.

```
IN      200     ; doc A
IN      201     ; doc B
LOAD    200     ; R <- A
COMPARE 201     ; so sanh A voi B
JUMPGT  400     ; neu A > B, den OUT A
OUT     201     ; nguoc lai xuat B
JUMP    401     ; bo qua nhanh xuat A
OUT     200     ; dia chi 400: xuat A
HALT            ; dia chi 401
```

Điều kiện

COMPARE cập nhật status flags; JUMPGT kiểm tra kết quả.

Hai nhánh

Một đường xuất B, đường còn lại nhảy tới lệnh xuất A.

Nhãn

Assembler hiện đại dùng symbolic labels thay cho 400 và 401.

Chương trình biến cùng một phần cứng thành nhiều công cụ

Một program là chuỗi lệnh có thứ tự mô tả các thao tác phần cứng.

PROGRAM

Tập hợp các lệnh được sắp xếp theo trình tự; mỗi lệnh mô tả một thao tác cơ bản mà phần cứng có thể thực hiện.

Phần cứng

Quy định tập lệnh mà máy có khả năng thực thi.

Phần mềm

Chọn và sắp xếp chuỗi lệnh để giải quyết một nhiệm vụ.

Ada Lovelace — 1843

Nhận ra Analytical Engine có thể được lập trình bằng thẻ đục lỗ.

Ba mức lập trình đánh đổi trừu tượng và kiểm soát

Mỗi mức cao hơn dễ biểu đạt hơn nhưng xa phần cứng hơn.

MACHINE CODE

Chuỗi bit do CPU thực thi trực tiếp.

Khó đọc, không di động giữa ISA và cho kiểm soát chi tiết nhất.

ASSEMBLY

Mnemonic và label ánh xạ gần một-một với lệnh máy.

Dễ đọc hơn binary nhưng vẫn phụ thuộc ISA.

HIGH-LEVEL LANGUAGE

C, C++, Java và Python cung cấp hàm, kiểu dữ liệu và cấu trúc điều khiển.

Một câu lệnh có thể thành nhiều lệnh máy.

Một vòng lặp bậc cao trở thành nhiều lệnh assembly

Ví dụ tính tổng từ 1 đến 10 ở hai mức trừu tượng.

C — High-level

```
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += i;
```

Ý nghĩa

Cấu trúc for biểu đạt khởi tạo, điều kiện, cập nhật và thân vòng lặp.

x86-64 assembly — chưa tối ưu

```
movl  $0, -4(%rbp)       ; sum = 0
movl  $1, -8(%rbp)       ; i = 1
loop:
cmpl  $10, -8(%rbp)
jg    .L2
movl  -4(%rbp), %eax
addl  -8(%rbp), %eax
movl  %eax, -4(%rbp)
addl  $1, -8(%rbp)
jmp   loop
.L2:
```

Compiler có thể loại bỏ toàn bộ vòng lặp

Tối ưu hóa khai thác ngữ nghĩa chương trình, không chỉ dịch từng dòng máy móc.

MACHINE CODE

Lệnh `movl $0, -4(%rbp)` có thể trở thành chuỗi byte:

`C7 45 FC 00 00 00 00`

Các trường

Opcode xác định phép toán; ModR/M chọn chế độ địa chỉ; displacement và immediate mang dữ liệu bổ sung.

CONSTANT FOLDING — -O2

Compiler nhận ra $1 + 2 + \cdots + 10 = 55$ tại compile time.

```
movl  $55, -4(%rbp)  ; sum = 55
```

Các mức trừu tượng có điểm mạnh và giới hạn khác nhau

Hệ thống thực tế thường kết hợp nhiều mức trong cùng một sản phẩm.

Tiêu chí	Mã máy	Hợp ngữ	Bậc cao
Mức trừu tượng	Không có	Thấp	Cao
Kiểm soát hardware	Hoàn toàn	Hoàn toàn	Hạn chế
Tính di động	Không có	Không có	Cao tương đối
Khả năng đọc	Rất thấp	Thấp	Cao
Tốc độ phát triển	Rất chậm	Chậm	Nhanh
Hiệu năng	Cao nếu viết tốt	Cao nếu viết tốt	Tùy compiler/runtime
Sử dụng	Firmware khởi động	Driver, ISR	Ứng dụng, hệ thống

Hệ thống lớn kết hợp nhiều mức trừu tượng

Điểm cân bằng được chọn riêng cho từng thành phần và yêu cầu.

Linux kernel

Viết chủ yếu bằng C; dùng inline assembly khi cần truy cập register đặc biệt.

Game engine

Dùng C++ cho logic; gọi shader bằng ngôn ngữ GPU chuyên dụng.

Python ecosystem

API bậc cao nhưng phần tính toán nặng có thể chạy trong thư viện C/C++ như NumPy hoặc TensorFlow.

Trừu tượng, kiểm soát và hiệu năng là một tam giác đánh đổi, không phải lựa chọn tuyệt đối.

06

TÓM TẮT VÀ ÔN TẬP

Kết nối mô hình phần cứng với cách chương trình được lưu trữ, dịch và thực thi.

Chương 1 • Kiến trúc máy tính • Nguyên lý Von Neumann

Thông điệp chính: hệ thống và phần cứng

Những khái niệm nền tảng cần giữ lại sau chương.

- **Programmable computer:** Hardware cung cấp khả năng; software quyết định hành vi.
- **Abstraction:** Sáu lớp che giấu dần độ phức tạp từ transistor đến ngôn ngữ bậc cao.
- **Von Neumann:** CPU, memory, I/O và bus dựa trên stored-program và thực thi tuần tự.
- **CPU:** CU điều phối; ALU tính toán; register giữ trạng thái tốc độ cao.
- **Memory và I/O:** Hierarchy, cache, virtual memory, interrupt và DMA giảm khoảng cách tốc độ.

Thông điệp chính: lệnh và lập trình

Từ bit trong memory đến cấu trúc điều khiển của chương trình.

- **FDE cycle:** PC, MAR, MDR và IR phối hợp để fetch, decode và execute lệnh.
- **ISA:** Định dạng lệnh và tập lệnh tạo giao diện giữa hardware và software.
- **RISC/CISC:** Hai hướng thiết kế khác nhau nhưng hiệu năng còn phụ thuộc toàn hệ thống.
- **Assembly:** Mnemonic và label biểu diễn gần với machine code của một ISA cụ thể.
- **High-level:** Compiler dịch và tối ưu các trừu tượng thành chuỗi lệnh máy hiệu quả.

Câu hỏi ôn tập 1–4

Tập trung vào Von Neumann, CPU, cache và chu kỳ lệnh.

1. Trình bày ba nguyên lý cốt lõi của mô hình Von Neumann và giải thích vì sao stored-program có tính cách mạng.

2. Mô tả chức năng của Program Counter, Instruction Register, Instruction Decoder và ALU.

3. Phân biệt temporal locality và spatial locality; giải thích vai trò của chúng trong cache.

4. Vẽ sơ đồ và giải thích ba pha Fetch–Decode–Execute.

Câu hỏi ôn tập 5–8

So sánh ISA, ngôn ngữ và các thành phần hệ thống.

5 So sánh RISC và CISC theo định dạng lệnh, số toán hạng, mật độ mã và cơ chế pipeline.

6 Vì sao ngôn ngữ bậc cao di động hơn hợp ngữ nhưng kiểm soát phần cứng thấp hơn?

7 Hệ thống máy tính gồm những thành phần cơ bản nào? Nêu chức năng của từng thành phần.

8 Trình bày các nguyên lý cốt lõi của kiến trúc Von Neumann.

Câu hỏi ôn tập 9–12

Làm rõ stored-program, register và memory hierarchy.

9 Giải thích stored-program concept và ý nghĩa đối với hoạt động của máy tính.

10 Nêu chức năng của Control Unit, ALU và các register trong CPU.

11 Trình bày vai trò của PC và IR trong quá trình thực thi chương trình.

12 Mô tả các mức chính trong memory hierarchy và đánh đổi tốc độ, dung lượng, chi phí.

Câu hỏi ôn tập 13–16

Kết nối I/O, chu kỳ lệnh và quá trình dịch chương trình.

13 I/O có vai trò gì? Controller và interrupt hỗ trợ CPU làm việc với thiết bị ngoại vi như thế nào?

14 Trình bày Fetch–Decode–Execute và hoạt động chính của CPU trong từng giai đoạn.

15 Phân biệt machine code, assembly language và high-level programming language.

16 Trình bày vai trò của compiler và assembler trong việc tạo dạng chương trình CPU có thể thực thi.

07

BÀI TẬP

Ước lượng chi phí dữ liệu và dịch các cấu trúc điều khiển bậc cao thành lệnh nhảy.

Hai phương pháp xử lý dữ liệu có chi phí I/O khác nhau

Mảng ban đầu trong tệp đã sắp xếp: $2, 4, 7, 9, 16$.

PHƯƠNG PHÁP 1

Thực hiện chèn và mọi thao tác liên quan trực tiếp trên thiết bị lưu trữ thứ cấp.

Chi phí được đo bằng số lần đọc và ghi đĩa.

PHƯƠNG PHÁP 2

Sao chép toàn bộ dữ liệu vào memory chính, xử lý trong memory, sau đó ghi kết quả trở lại secondary storage.

Bài tập: ước lượng số thao tác đọc và ghi

Điền chi phí cho mỗi chuỗi phần tử được chèn vào tệp.

Phần tử cần thêm	Phương pháp 1	Phương pháp 2
1	_____	_____
8	_____	_____
18	_____	_____
1, 8, 18	_____	_____
18, 8, 1	_____	_____

Selection sort: đếm chi phí trên một ví dụ

Với mảng 12, 4, 27, 9, tổng chi phí là 9 lần đọc và 6 lần ghi.

```
<12 4 27 9>  -> doc 4 lan; min = 4
4 12 27 9    -> hoan doi -> 2 lan ghi

4 <12 27 9>  -> doc 3 lan; min = 9
4 9 27 12    -> hoan doi -> 2 lan ghi

4 9 <27 12>  -> doc 2 lan; min = 12
4 9 12 27    -> hoan doi -> 2 lan ghi
```

Mỗi lượt

Đọc phần chưa sắp xếp để tìm phần tử nhỏ nhất.

Hoán đổi

Tính hai lần ghi cho mỗi thao tác swap.

Bỏ qua

Không tính các lần đọc tạm thời ngoài quy ước của bài.

Bài tập: ước lượng chi phí sắp xếp

Áp dụng hai phương pháp lưu trữ cho bốn trạng thái dữ liệu.

Mảng	Phương pháp 1	Phương pháp 2
2, 4, 7, 9, 16	_____	_____
16, 9, 7, 4, 2	_____	_____
2, 4, 7, 19, 6	_____	_____
2, 19, 4, 7, 6	_____	_____

Thách thức dữ liệu lớn: memory chỉ giữ ba phần tử

Tối thiểu hóa tổng số lần đọc và ghi cho mảng tám phần tử.

$$2,\ 19,\ 4,\ 7,\ 6,\ 8,\ 24,\ 17$$

Ràng buộc

Main memory chỉ chứa tối đa ba phần tử; phần còn lại nằm trên secondary storage.

Yêu cầu

Xây dựng giải pháp và trình bày đầy đủ các bước trung gian.

Mở rộng

Đề xuất giải pháp tổng quát cho mảng và kích thước memory bất kỳ.

Năm chỉ thị jump dùng trong bài tập sinh mã

Sử dụng nhãn nhất quán và bảo toàn đúng luồng điều khiển.

Chỉ thị	Ý nghĩa
`JEO a, L`	Nhảy đến L nếu a bằng 0
`JGO a, L`	Nhảy đến L nếu a dương
`JLO a, L`	Nhảy đến L nếu a âm
`JLEO a, L`	Nhảy đến L nếu a không dương
`JMP L`	Nhảy không điều kiện đến L

Sinh mã: if một nhánh và if–else

Điền chuỗi jump thích hợp; S1 và S2 được xem là các chuỗi lệnh hợp lệ.

IF một nhánh

```
if (a > 0)
    S1;

Assembly:
-------------------------
-------------------------
-------------------------
```

IF–ELSE hai nhánh

```
if (a <= 0)
    S1;
else
    S2;

Assembly:
-------------------------
-------------------------
-------------------------
```

Sinh mã: while có break

Điểm thoát của break phải nhảy ra ngoài toàn bộ vòng lặp.

```
while (a != 0) {
    if (a > 0)
        break;
    else
        S;
}
```

Loop test

Nếu a bằng 0, nhảy tới nhãn kết thúc.

Break test

Nếu a dương, cũng nhảy tới cùng nhãn kết thúc.

Back edge

Sau S, JMP trở lại nhãn kiểm tra đầu vòng lặp.

Assembly:

Sinh mã: do–while có continue

Continue phải chuyển điều khiển tới bước kiểm tra điều kiện cuối vòng.

```
do {
    if (a == 0)
        continue;
    else
        S;
} while (a < 0);
```

Body first

Thân do–while luôn chạy ít nhất một lần.

Continue label

Nếu a bằng 0, nhảy thẳng tới phép kiểm tra a âm.

Repeat

Nếu a âm, jump về đầu thân; ngược lại kết thúc.

Assembly:

Sinh mã: repeat–until

Vòng lặp kết thúc khi điều kiện until trở thành đúng.

```
repeat
    if (a >= 0)
        S;
until (a > 0)
```

Điều kiện trong thân

S chỉ chạy khi a không âm.

Exit condition

Nếu a dương, kết thúc vòng lặp.

Repeat condition

Nếu a không dương, jump về đầu repeat.

Assembly:

Nguồn nội dung và hình ảnh

Toàn bộ nội dung giảng dạy được phát triển từ thư mục Chapter 1 được cung cấp.

Tài liệu nguồn

intro.tex — Chương 1: Kiến trúc máy tính và nguyên lý Von Neumann.

Hình ảnh được sử dụng

eniac-wikipedia.jpg • edvac.png • von-neumann-architecture.png
basic-computer.png • alu-block.png • memory-hierarchy.png
Các sơ đồ còn lại được dựng lại bằng TikZ từ nội dung trong **intro.tex**.

CHƯƠNG 1

Kiến trúc máy tính và nguyên lý Von Neumann

Từ phần cứng vật lý đến chương trình có thể thực thi.